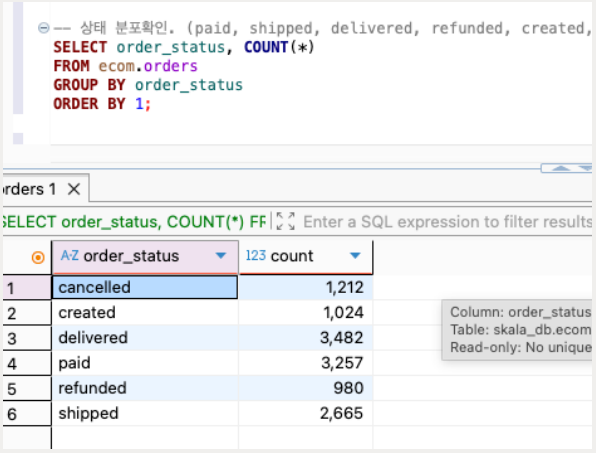
스마트 데이터 이해 및 활용 - 종합실습 4 제출물

날짜	2026년 7월 30일
태그	

이름 김민정 사번 P282 반 9반

Q1) 지난 한 달간 실제 팔린 총 금액 보기 (paid+shipped+delivered)

우선 ***실제 상태값 분포*** 를 확인했습니다.
주어진 3개의 값 paid, shipped, delivered 를 제외하고, refunded, created, cancelled 가 추가로 있는 것을 확인했습니다.



- **paid , shipped, delivered ⇒ 결제가 실제로 이루어진 값들**
- **created (=아직 결제 전), cancelled (=안 팔린 것), refunded(환불. 순매출 아님)**

이기 때문에 위 세 값만 필터링해서 합산하는 것으로 생각했습니다.

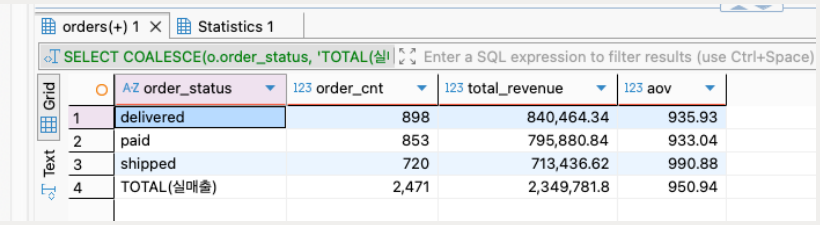
쿼리

```
-- 확인된 paid, shipped, delivered, refunded, created, cancelled 모든 값을 포함했습니다.
SELECT
    COALESCE(o.order_status, 'TOTAL(실매출)')              AS order_status,  -- 상태
별 값, ROLLUP 합계 행은 'TOTAL(실매출)'로 표시
    COUNT(DISTINCT o.order_id)                             AS order_cnt,     -- 상태
별(및 합계) 주문 건수, 중복 order_id 제거
    SUM(oi.line_total)                                     AS total_revenue, -- 상태
별(및 합계) 실매출 금액(수량*단가-할인)
    ROUND(SUM(oi.line_total) / NULLIF(COUNT(DISTINCT o.order_id), 0), 2) AS aov -- 상
태별 평균 주문금액, 0나눔 방지
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')            -- 실제 매출로 인정
하는 상태만 필터 (created/cancelled/refunded 제외)
  AND o.order_ts >= date_trunc('month', now() - interval '1 month')  -- 지난 달력월 시작
  AND o.order_ts <  date_trunc('month', now())                       -- 이번 달 시작 전
까지 (지난 달 전체)
```

```
GROUP BY ROLLUP (o.order_status);        -- paid/shipped/delivered 소계 3줄 + 맨 아래 실
매출 합계 1줄 생성
```

결과

"실제 팔린 총 금액"의 기준이 되는 `paid`, `shipped`, `delivered` 세 상태만 필터링해서, 지난 한 달간 각 상태별 주문 건수·금액·평균 주문금액(AOV)을 보여주고 맨 아래에 세 상태를 합친 최종 실매출 합계를 함께 산출했습니다.
`created`(미결제), `cancelled`(취소), `refunded`(환불)는 제외했기 때문에, 이 결과의 `TOTAL(실매출)` 행이 곧 Q1의 최종 답(지난 한 달간 실제 팔린 총 금액)입니다.



	order_status	order_cnt	total_revenue	aov
1	delivered	898	840,464.34	935.93
2	paid	853	795,880.84	933.04
3	shipped	720	713,436.62	990.88
4	TOTAL(실매출)	2,471	2,349,781.8	950.94

Q2) 월별로 몇 개 주문이 있었고, 얼마 벌었는지, 주문당 평균 금액은 얼마인지

Q1에서 "실제 매출"은 paid/shipped/delivered 세 상태만 인정하기로 정의했었는데, Q2도 "얼마 벌었는지"를 묻는 거라 같은 기준을 그대로 가져가면 된다고 판단했습니다. 다만 이번엔 특정 한 달이 아니라 "월별로" 나눠서 봐야 하니까, 날짜를 필터링하는 게 아니라 order_ts를 월 단위로 묶는(bucketing) 방식으로 접근했습니다.

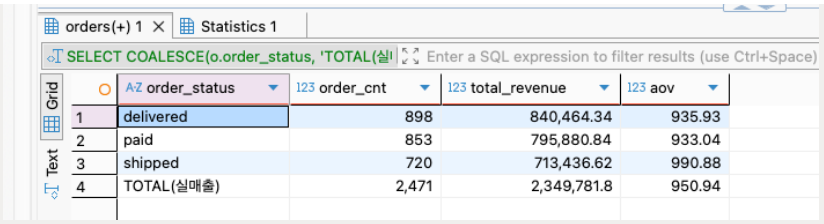
- date_trunc('month', order_ts)로 각 주문의 날짜를 그 달의 1일로 표기 후, to_char 를 사용하여 “YYYY-MM” 형태로 변환
- 이 값으로 GROUP BY를 하면 자연스럽게 월별 집계가 됨
- 주문 건수는 order_items와 조인하면서 행이 뻥튀기되니까 COUNT(DISTINCT order_id)로 세야 함
- AOV는 Q1과 동일하게 `SUM(line_total) / COUNT(DISTINCT order_id)`, 0으로 나뉘는 것 방지 위해 `NULLIF` 사용

쿼리

```
SELECT
    to_char(date_trunc('month', o.order_ts), 'YYYY-MM')     AS order_month,   -- "YYY
Y-MM" 문자열로 표시
    COUNT(DISTINCT o.order_id)                                AS order_cnt,     -- 월
별 주문 건수, 중복 order_id 제거
    SUM(oi.line_total)                                        AS total_revenue, -- 월
별 실매출(수량*단가-할인 합)
    ROUND(SUM(oi.line_total) / NULLIF(COUNT(DISTINCT o.order_id), 0), 2) AS aov -- 월
별 평균 주문금액, 0나눔 방지
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')  -- 실제 매출로 인정하는 상태만
GROUP BY date_trunc('month', o.order_ts)
ORDER BY order_month;
```

결과

date_trunc('month', ...)로 모든 주문을 월 단위 버킷으로 묶고 그 기준으로 GROUP BY해서, 달마다 주문 건수·매출·AOV를 한 줄씩 뽑았습니다. 날짜를 필터링한 게 아니라 그룹핑한 거라 데이터에 있는 전체 기간의 월별 추이를 한 번에 볼 수 있고, 매출 기준은 Q1과 동일하게 paid/shipped/delivered만 인정해 취소·환불·미결제 건은 제외했습니다.



Q3) 최근 90일 카테고리 Top10

categories 총 14개

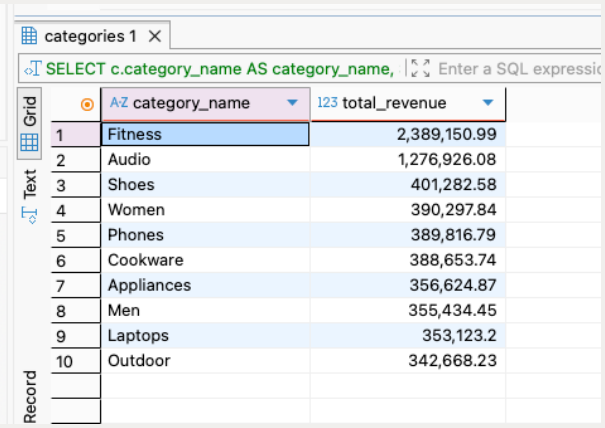
- Top10을 가르는 기준 = 카테고리별 실매출(`SUM(line_total)`)
- 카테고리는 리프 레벨(`categories.category_name`) 노출
- 매출 기준은 이전과 동일하게 paid/shipped/delivered만 인정
- 최근 90일은 롤링 윈도우(`now() - interval '90 days'`)

쿼리

```
SELECT
    c.category_name                                  AS category_name,    -- 카테고리
이름
    SUM(oi.line_total)                               AS total_revenue     -- Top10을
가르는 기준: 카테고리별 실매출
FROM ecom.order_items oi
JOIN ecom.orders o     ON o.order_id = oi.order_id
JOIN ecom.products p   ON p.product_id = oi.product_id
JOIN ecom.categories c ON c.category_id = p.category_id    -- 상품의 카테고리와 매칭
WHERE o.order_status IN ('paid', 'shipped', 'delivered')   -- 실제 매출 기준
  AND o.order_ts >= now() - interval '90 days'             -- 최근 90일 롤링 윈도우
GROUP BY c.category_name
ORDER BY total_revenue DESC
LIMIT 10;
```

결과

최근 90일간 `paid` / `shipped` / `delivered` 상태의 주문만 대상으로 카테고리별 실매출을 합산하고, 그 매출액을 기준으로 상위 10개 카테고리만 뽑았습니다. Top10을 결정하는 기준은 `total_revenue`(카테고리별 합산 매출) 입니다.



Q4) 제품별 누적매출 RANK() Top20

데이터에 있는 전체 기간 동안 그 제품이 벌어들인 총 매출을 말하는 거라고 판단해서, 날짜 필터는 아예 넣지 않았습니다. 매출 인정 기준(paid/shipped/delivered)은 이전 질문들과 동일하게 유지했습니다. 그리고 `order_items.unit_price`가 이미 주문 시점의 가격(SCD2 가격이력 반영값)을 그대로 저장하고 있어서, `product_prices`를 따로 join할 필요가 없다는 것도 확인했습니다.

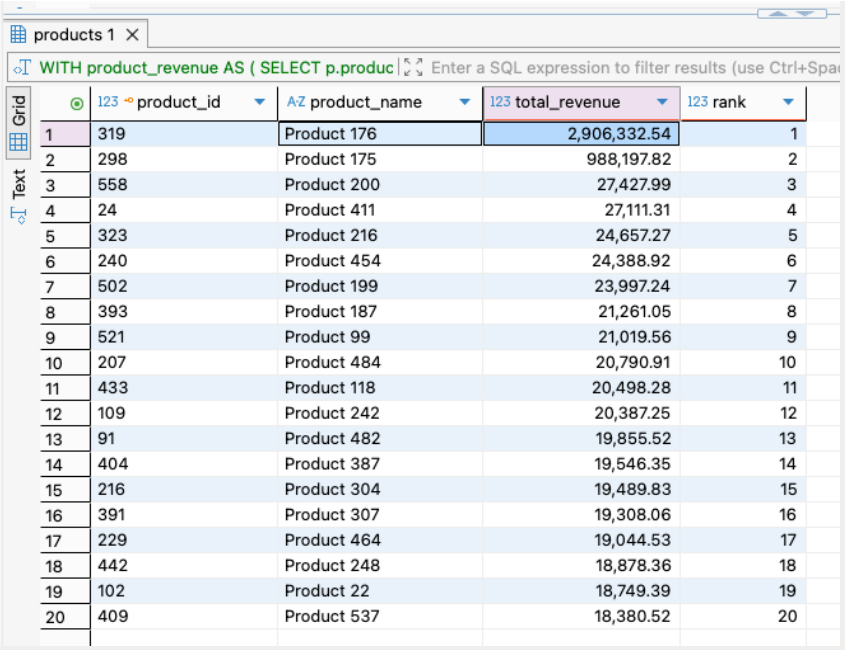
- 제품별로 먼저 전체 기간 매출을 `GROUP BY`로 집계
- 그 다음 집계된 결과 위에 `RANK() OVER (ORDER BY total_revenue DESC)`를 적용해서 순위를 매김
- `RANK()`를 쓴 이유(질문에 명시된 함수)는 동점 처리 때문
  → 매출이 같은 제품이 있으면 같은 순위를 주고 다음 순위를 건너뜀 (`ROW_NUMBER()`였다면 동점이어도 강제로 순번을 나눠버려서 의미가 달라짐)
- 그래서 자르는 것도 `LIMIT 20`이 아니라 `WHERE rnk <= 20`으로 처리
  → 20등이 동점으로 여러 개면 그 동점 제품들을 전부 보여주는 게 RANK()의 취지에 맞다고 판단

쿼리

```
WITH product_revenue AS (
    SELECT
        p.product_id,
        p.product_name,
        SUM(oi.line_total) AS total_revenue                    -- 제품별 전체 기간 누적 실
매출
    FROM ecom.order_items oi
    JOIN ecom.orders o   ON o.order_id = oi.order_id
    JOIN ecom.products p ON p.product_id = oi.product_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')  -- 실제 매출 기준(Q1과 동
일)
    GROUP BY p.product_id, p.product_name
),
ranked AS (
    SELECT
        product_id,
        product_name,
        total_revenue,
        RANK() OVER (ORDER BY total_revenue DESC) AS rank      -- 누적매출 기준 순위, 동
점은 같은 순위
    FROM product_revenue
)
SELECT product_id, product_name, total_revenue, rank
FROM ranked
WHERE rank <= 20                                               -- 20등 동점 제품까지 전부
포함
ORDER BY rank;
```

결과

전체 기간의 `paid` / `shipped` / `delivered` 주문만 대상으로 제품별 누적 매출을 먼저 합산한 뒤, `RANK()` 윈도우 함수로 매출 순위를 매겨 상위 20위까지 뽑았습니다. `ROW_NUMBER()` 대신 `RANK()`를 쓴 건 동점 매출을 같은 순위로 처리하기 위해서고, 그래서 자르는 조건도 `LIMIT 20`이 아니라 `WHERE rnk <= 20`으로 둬서 20위가 동점이면 그 제품들을 누락 없이 보여줍니다.



products 1

WITH product_revenue AS (SELECT p.produc | Enter a SQL expression to filter results (use Ctrl+Spa

Grid | Text

	product_id	product_name	total_revenue	rank
1	319	Product 176	2,906,332.54	1
2	298	Product 175	988,197.82	2
3	558	Product 200	27,427.99	3
4	24	Product 411	27,111.31	4
5	323	Product 216	24,657.27	5
6	240	Product 454	24,388.92	6
7	502	Product 199	23,997.24	7
8	393	Product 187	21,261.05	8
9	521	Product 99	21,019.56	9
10	207	Product 484	20,790.91	10
11	433	Product 118	20,498.28	11
12	109	Product 242	20,387.25	12
13	91	Product 482	19,855.52	13
14	404	Product 387	19,546.35	14
15	216	Product 304	19,489.83	15
16	391	Product 307	19,308.06	16
17	229	Product 464	19,044.53	17
18	442	Product 248	18,878.36	18
19	102	Product 22	18,749.39	19
20	409	Product 537	18,380.52	20

Q5) 고객이 얼마나 최근에, 얼마나 자주, 얼마나 많이 샀는지 (최근성/빈도/금액)

어떤 것을 얼마나 자주 / 일별할 수도 있고 월별 할 수도 있고 그 기준은 자유롭게

매출 인정 기준은 계속 써온 paid/shipped/delivered 로 통일했고, 세 지표를 각각 이렇게 정의했습니다

- R(최근성): 일(day) 단위
- F(빈도): 월(month) 단위로 통일 - "구매 건수 ÷ 구매가 있었던 달 수"
- M(금액): 전체 기간 누적 합계

다만 빈도의 경우, 어제 가입해서 1번 산 고객이랑 4개월 동안 1번산 고객이 똑같이 빈도는 1이지만 실제로 전자가 더 자주 사는 고객이기 때문에 기준을 다음과 같이 정했습니다.
데이터의 주문 기간이 최근 약 120일 기준이라서 일 단위로 나누면 대부분 0.0x 수준으로 나와서 해석이 어렵고, 월 단위가 훨씬 직관적이라고 생각해 월 기준으로 정했습니다.

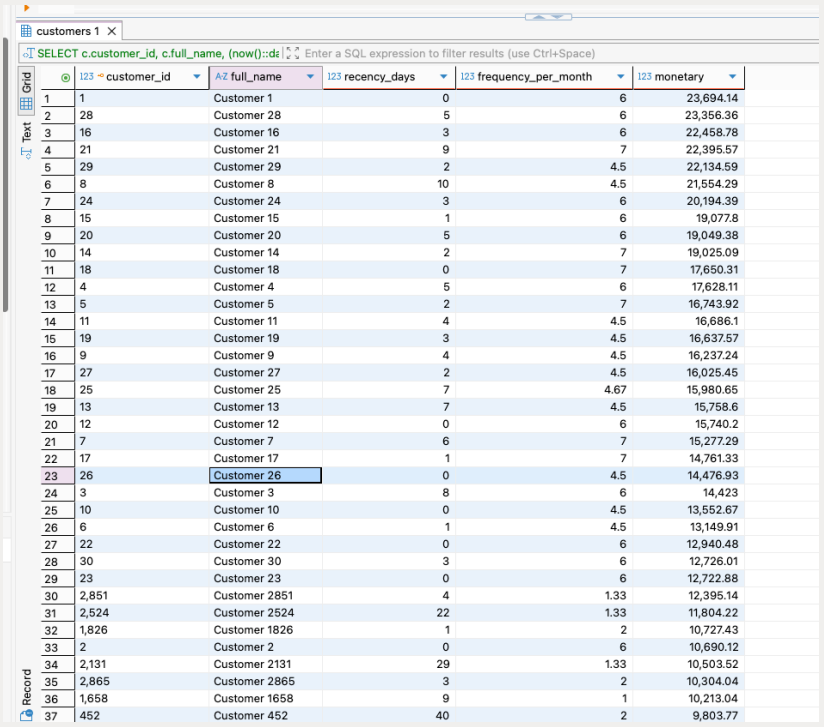
- **빈도(F) 기준**: **월평균 구매 횟수** = 구매 건수 ÷ 구매가 있었던 서로 다른 달의 개수
- R(최근성): 일 단위 - `now() - MAX(order_ts)`, 마지막 구매일로부터 경과일수
- M(금액): 전체 기간 누적 합계 - `SUM(line_total)`
- 정렬은 금액(M) 큰 순

쿼리

```
SELECT
    c.customer_id,
    c.full_name,
    (now()::date - MAX(o.order_ts)::date)                    AS recency_days,
-- R: 마지막 구매일로부터 경과일수
    ROUND(
        COUNT(DISTINCT o.order_id)::numeric
        / COUNT(DISTINCT date_trunc('month', o.order_ts)), 2
    )                                                          AS frequency_per_month,
-- F: 월 기준 구매 빈도(=구매건수/구매한 달 수)
```

```
    SUM(oi.line_total)                                    AS monetary
-- M: 누적 구매액
FROM ecom.customers c
JOIN ecom.orders o       ON o.customer_id = c.customer_id
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')  -- 실제 매출 기준(Q1과 동일)
GROUP BY c.customer_id, c.full_name
ORDER BY monetary DESC;
```

결과



customers 1

SELECT c.customer_id, c.full_name, (now()::da Enter a SQL expression to filter results (use Ctrl+Space)

	customer_id	full_name	recency_days	frequency_per_month	monetary
1	1	Customer 1	0	6	23,694.14
2	28	Customer 28	5	6	23,356.36
3	16	Customer 16	3	6	22,458.78
4	21	Customer 21	9	7	22,395.57
5	29	Customer 29	2	4.5	22,134.59
6	8	Customer 8	10	4.5	21,554.29
7	24	Customer 24	3	6	20,194.39
8	15	Customer 15	1	6	19,077.8
9	20	Customer 20	5	6	19,049.38
10	14	Customer 14	2	7	19,025.09
11	18	Customer 18	0	7	17,650.31
12	4	Customer 4	5	6	17,628.11
13	5	Customer 5	2	7	16,743.92
14	11	Customer 11	4	4.5	16,686.1
15	19	Customer 19	3	4.5	16,637.57
16	9	Customer 9	4	4.5	16,237.24
17	27	Customer 27	2	4.5	16,025.45
18	25	Customer 25	7	4.67	15,980.65
19	13	Customer 13	7	4.5	15,758.6
20	12	Customer 12	0	6	15,740.2
21	7	Customer 7	6	7	15,277.29
22	17	Customer 17	1	7	14,761.33
23	26	Customer 26	0	4.5	14,476.93
24	3	Customer 3	8	6	14,423
25	10	Customer 10	0	4.5	13,552.67
26	6	Customer 6	1	4.5	13,149.91
27	22	Customer 22	0	6	12,940.48
28	30	Customer 30	3	6	12,726.01
29	23	Customer 23	0	6	12,722.88
30	2,851	Customer 2851	4	1.33	12,395.14
31	2,524	Customer 2524	22	1.33	11,804.22
32	1,826	Customer 1826	1	2	10,727.43
33	2	Customer 2	0	6	10,690.12
34	2,131	Customer 2131	29	1.33	10,503.52
35	2,865	Customer 2865	3	2	10,304.04
36	1,658	Customer 1658	9	1	10,213.04
37	452	Customer 452	40	2	9,803.77

Q6) 첫 구매 후 30일 내 재구매율

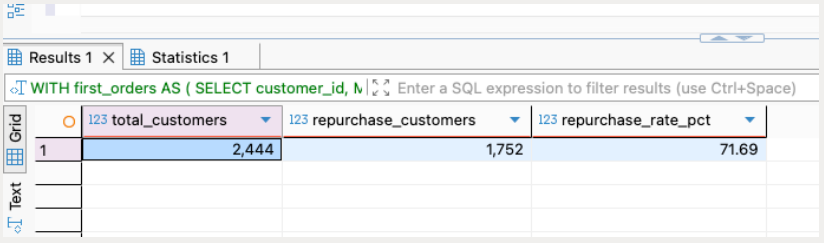
- 재구매를 상품 단위가 아니라 주문 단위로 정의했습니다.
- 재구매 판단 기준은 "고객이 실제로 결제 행위를 한 번 더 했는가"이기 때문에, `refunded`(환불)도 포함시켰습니다.
- 환불은 애초에 결제(paid)가 먼저 일어나야만 나올 수 있는 상태라, 그 시점엔 분명히 구매 행위가 있었던 것이고 이후에 취소된 것뿐이라고 봤습니다.
- `created`(아직 결제 전)와 `cancelled`(결제 자체가 이뤄지지 않음)는 실제 구매 행위가 없었던 경우라 필터링했습니다.
- 첫 구매 시점(first_order_ts)은 위 기준을 만족하는 고객별 최초 주문 시점으로 잡았습니다.
- 재구매 여부는 같은 고객의 다른 주문이 첫 구매 이후 30일 이내에 존재하는지로 판단했습니다.
- 재구매율은 재구매한 고객 수를 첫 구매를 한 전체 고객 수로 나눈 비율입니다.

쿼리

```
-- Q6) 첫 구매 후 30일 내 재구매율
WITH first_orders AS (
    SELECT
        customer_id,
```

```
        MIN(order_ts) AS first_order_ts                                -- 고객별 첫 "결제 완
료" 시점
    FROM ecom.orders
    WHERE order_status NOT IN ('created', 'cancelled')                 -- 결제가 실제로 이뤄진
상태만 (refunded 포함)
    GROUP BY customer_id
),
repurchase AS (
    SELECT DISTINCT f.customer_id                                       -- 30일 내 재구매(상품
무관) 고객만
    FROM first_orders f
    JOIN ecom.orders o
      ON o.customer_id = f.customer_id
     AND o.order_status NOT IN ('created', 'cancelled')                -- 재구매도 동일 기준
     AND o.order_ts > f.first_order_ts                                  -- 첫 구매 "이후"
     AND o.order_ts <= f.first_order_ts + interval '30 days'            -- 30일 이내
)
SELECT
    (SELECT COUNT(*) FROM first_orders)                                AS total_customers,
-- 첫 구매한 전체 고객 수
    (SELECT COUNT(*) FROM repurchase)                                  AS repurchase_custom
ers,  -- 30일 내 재구매한 고객 수
    ROUND(
        100.0 * (SELECT COUNT(*) FROM repurchase)
        / NULLIF((SELECT COUNT(*) FROM first_orders), 0), 2
    )                                                                    AS repurchase_rate
_pct;   -- 재구매율(%)
```

결과



Q7) 재고가 임계치보다 낮은 상품 찾아내기 (곧 품절될 위험이 있는 상품은?)

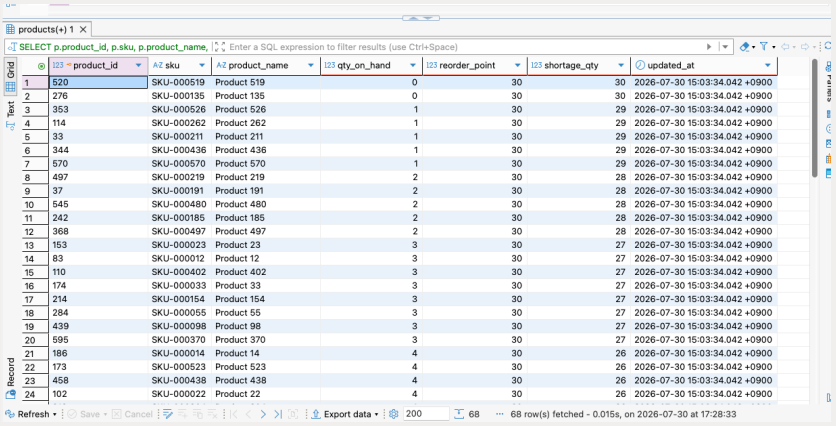
재고 관리는 `inventory` 테이블의 `qty_on_hand`(현재 재고)와 `reorder_point`(재주문 임계치)를 비교해서 판단했습니다.
재고가 임계치에 도달한 순간부터 이미 재주문이 필요한 상태라고 보고, `qty_on_hand < reorder_point`가 아니라 `<=` 조건으로 임계치와 같은 경우도 위험군에 포함시켰습니다. 이미 판매하지 않는 상품(`active = false`)까지 재고 부족으로 잡으면 실무적으로 의미가 없다고 판단해 활성 상품만 대상으로 필터링했습니다.
정렬 기준은 얼마나 급한지를 보여주기 위해 부족 수량(`reorder_point - qty_on_hand`)이 큰 순으로 두었습니다. 이 데이터에서는 reorder_point가 모두 30으로 고정돼 있지만, 향후 상품마다 임계치가 달라지는 상황에서도 동일한 기준이 적용되도록 절대 재고량이 아닌 부족분 기준으로 정렬했습니다.

쿼리

```
SELECT
    p.product_id,
    p.sku,
    p.product_name,
    i.qty_on_hand,                                  -- 현재 재고
    i.reorder_point,                                -- 재주문 임계치
```

```sql
    (i.reorder_point - i.qty_on_hand) AS shortage_qty,  -- 부족 수량 (클수록 위험)
    i.updated_at
FROM ecom.inventory i
JOIN ecom.products p ON p.product_id = i.product_id
WHERE i.qty_on_hand <= i.reorder_point   -- 임계치 도달~이하는 재주문 필요 상태
  AND p.active = true                    -- 판매 중인 상품만 대상
ORDER BY shortage_qty DESC;
```

결과



	product_id	sku	product_name	qty_on_hand	reorder_point	shortage_qty	updated_at
1	520	SKU-000519	Product 519	0	30	30	2026-07-30 15:03:34.042 +0900
2	276	SKU-000135	Product 135	0	30	30	2026-07-30 15:03:34.042 +0900
3	353	SKU-000526	Product 526	1	30	29	2026-07-30 15:03:34.042 +0900
4	114	SKU-000262	Product 262	1	30	29	2026-07-30 15:03:34.042 +0900
5	33	SKU-000211	Product 211	1	30	29	2026-07-30 15:03:34.042 +0900
6	344	SKU-000436	Product 436	1	30	29	2026-07-30 15:03:34.042 +0900
7	570	SKU-000570	Product 570	1	30	29	2026-07-30 15:03:34.042 +0900
8	497	SKU-000219	Product 219	2	30	28	2026-07-30 15:03:34.042 +0900
9	37	SKU-000191	Product 191	2	30	28	2026-07-30 15:03:34.042 +0900
10	545	SKU-000480	Product 480	2	30	28	2026-07-30 15:03:34.042 +0900
11	242	SKU-000185	Product 185	2	30	28	2026-07-30 15:03:34.042 +0900
12	368	SKU-000497	Product 497	2	30	28	2026-07-30 15:03:34.042 +0900
13	153	SKU-000023	Product 23	3	30	27	2026-07-30 15:03:34.042 +0900
14	83	SKU-000012	Product 12	3	30	27	2026-07-30 15:03:34.042 +0900
15	110	SKU-000402	Product 402	3	30	27	2026-07-30 15:03:34.042 +0900
16	174	SKU-000033	Product 33	3	30	27	2026-07-30 15:03:34.042 +0900
17	214	SKU-000154	Product 154	3	30	27	2026-07-30 15:03:34.042 +0900
18	284	SKU-000055	Product 55	3	30	27	2026-07-30 15:03:34.042 +0900
19	439	SKU-000098	Product 98	3	30	27	2026-07-30 15:03:34.042 +0900
20	595	SKU-000370	Product 370	3	30	27	2026-07-30 15:03:34.042 +0900
21	186	SKU-000014	Product 14	4	30	26	2026-07-30 15:03:34.042 +0900
22	173	SKU-000523	Product 523	4	30	26	2026-07-30 15:03:34.042 +0900
23	458	SKU-000438	Product 438	4	30	26	2026-07-30 15:03:34.042 +0900
24	102	SKU-000022	Product 22	4	30	26	2026-07-30 15:03:34.042 +0900

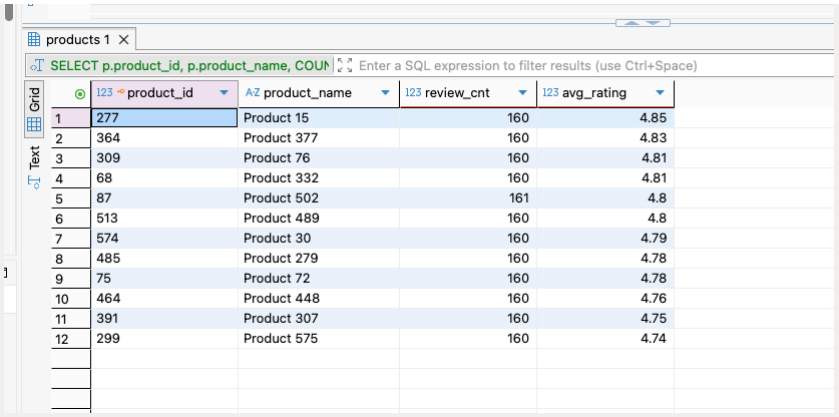
Q8) 리뷰 4.5↑ & 50개↑ 효자상품 (리뷰가 많고 평가도 좋은 효자상품 찾기)

효자상품 판별 기준은 질문에 명시된 대로 평균 평점 4.5 이상, 리뷰 개수 50개 이상 두 조건을 모두 만족하는 상품으로 정의했습니다.
상품별로 리뷰를 집계한 뒤 `HAVING` 절로 두 조건을 동시에 걸어서, 리뷰가 많아도 평점이 낮거나 평점이 높아도 리뷰가 적은 상품은 제외되도록 했습니다.
정렬은 평점이 높은 순, 동점이면 리뷰 개수가 많은 순으로 두어 신뢰도가 높은 상품이 위로 오도록 했습니다.

쿼리

```sql
SELECT
    p.product_id,
    p.product_name,
    COUNT(r.review_id)              AS review_cnt,    -- 리뷰 개수
    ROUND(AVG(r.rating), 2)         AS avg_rating     -- 평균 평점
FROM ecom.reviews r
JOIN ecom.products p ON p.product_id = r.product_id
GROUP BY p.product_id, p.product_name
HAVING COUNT(r.review_id) >= 50                       -- 리뷰 개수 조건
   AND AVG(r.rating) >= 4.5                           -- 평균 평점 조건
ORDER BY avg_rating DESC, review_cnt DESC;
```

결과



Q9) 쿠폰 사용 영향(쿠폰을 쓴 주문과 안 쓴 주문의 평균 주문 금액 비교)

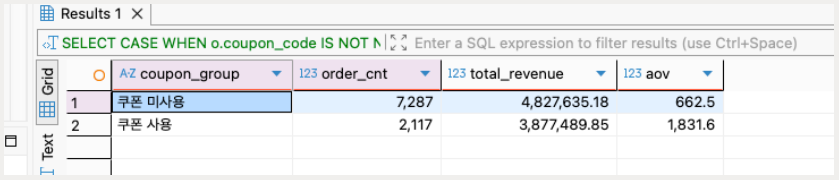
10프로 할인 쿠폰을 썼을 때의 주문과 비교

쿠폰 사용 여부는 `orders.coupon_code`가 NULL인지 아닌지로 구분했습니다.
매출 인정 기준은 계속 써온 대로 paid/shipped/delivered로 통일해서, 실제로 완료된 주문끼리만 비교했습니다. 두 그룹으로 나눠 주문 건수, 총매출, 평균 주문금액(AOV)을 함께 뽑아서 단순히 평균값 하나만 보는 게 아니라 표본 크기도 같이 확인할 수 있게 했습니다.

쿼리

```
SELECT
    CASE WHEN o.coupon_code IS NOT NULL THEN '쿠폰 사용' ELSE '쿠폰 미사용' END AS coupo
n_group,
    COUNT(DISTINCT o.order_id)                                    AS order_cnt,      -
- 그룹별 주문 건수
    SUM(oi.line_total)                                            AS total_revenue, -
- 그룹별 총매출(할인 반영 후)
    ROUND(SUM(oi.line_total) / NULLIF(COUNT(DISTINCT o.order_id), 0), 2) AS aov      -
- 그룹별 평균 주문금액
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')          -- 실제 매출 기준(Q1과
동일)
GROUP BY (o.coupon_code IS NOT NULL)
ORDER BY coupon_group;
```

결과



Q10) 상위 1% 고객의 최근 60일 매출

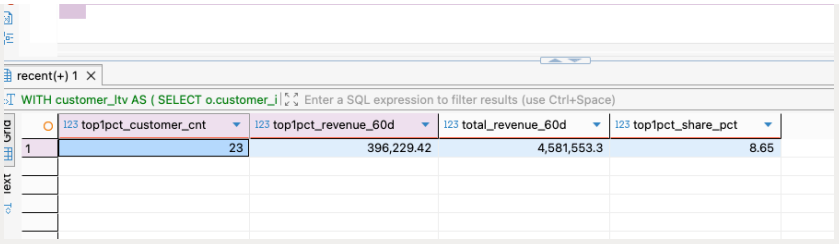
기준: 매출 기준으로 봐도 됨 얼마나 우리 사이트에 왓는지 자주 들어왔는지 기준ㅇ 이기준도 자유입 (자주 샀다. 자주 들어왔다. 이 두개에 따라 쿼리 를 짰더니 이렇게 만들어 졌고, .. 그 중에~)

구매 빈도 기준과 누적 매출 기준 두 가지로 각각 상위 1%를 뽑아 비교해봤는데, orders 테이블을 들여다보니 customer_id 1~30번대 고객들이 최근 60일 안에 주문이 유독 많이 몰려있고 결제 상태도 대부분 완료돼 있어서, 구매 빈도와 매출 두 지표 모두에서 이 고객군이 겹치게 최상위권으로 나왔습니다. 두 기준이 같은 고객군을 가리켰기 때문에, 최근 60일 매출을 직접 묻는 질문에 맞춰 누적 매출 기준으로 정했습니다.
전체 고객이 3000명이라 상위 1%는 30명 정도가 되는데, 실제로 앞서 확인한 고객군 규모와도 비슷하게 맞아떨어져서 이 기준으로 진행해도 되겠다고 판단했습니다.
상위 1% 판별은 고객별 누적 매출을 구한 뒤 NTILE(100)으로 100구간을 나눠, 가장 매출이 높은 1구간을 상위 1%로 정의했습니다.

쿼리

```sql
-- Q10) 상위 1% 고객(누적 매출 기준)의 최근 60일 매출
WITH customer_ltv AS (
    SELECT
        o.customer_id,
        SUM(oi.line_total) AS total_revenue          -- 고객별 전체 기간 누적 매출
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY o.customer_id
),
ranked AS (
    SELECT
        customer_id,
        NTILE(100) OVER (ORDER BY total_revenue DESC) AS pct_bucket  -- 누적 매출 기준
100분위, 1구간 = 상위 1%
    FROM customer_ltv
),
recent AS (
    SELECT
        o.customer_id,
        oi.line_total
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
      AND o.order_ts >= now() - interval '60 days'    -- 최근 60일 롤링 윈도우
)
SELECT
    COUNT(DISTINCT r.customer_id) FILTER (WHERE rk.pct_bucket = 1)
AS top1pct_customer_cnt,    -- 상위 1% 고객 수
    SUM(r.line_total)             FILTER (WHERE rk.pct_bucket = 1)
AS top1pct_revenue_60d,     -- 상위 1%의 최근 60일 매출
    SUM(r.line_total)
AS total_revenue_60d,       -- 전체 고객의 최근 60일 매출
    ROUND(
        100.0 * SUM(r.line_total) FILTER (WHERE rk.pct_bucket = 1)
        / NULLIF(SUM(r.line_total), 0), 2
    )
AS top1pct_share_pct        -- 상위 1%의 매출 기여 비중(%)
FROM recent r
JOIN ranked rk ON rk.customer_id = r.customer_id;
```

결과

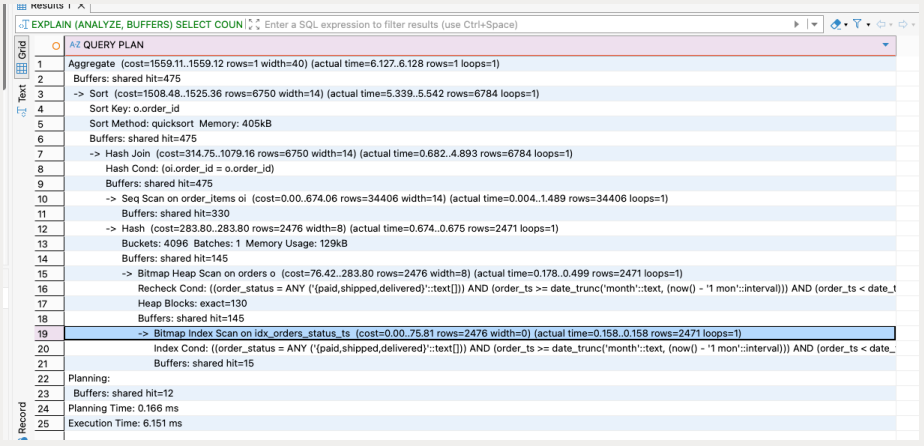


Q11) 0으로 나누어도 에러 안 나는 나눗셈 함수 써보기 → 안전하게 평균 계산하기 (0으로 나누기 방지)

일반 나눗셈은 분모가 0이면 division by zero 에러가 나서, 보통 `NULLIF(분모, 0)`으로 매번 방어해왔습니다.
0으로 나눠서 에러 안 나는 나눗셈 함수 → 분모가 0이거나 NULL이면 에러 대신 NULL을 반환하도록 만들어서, 호출하는 쪽에서 매번 예외처리를 반복하지 않아도 되게 했습니다.

쿼리

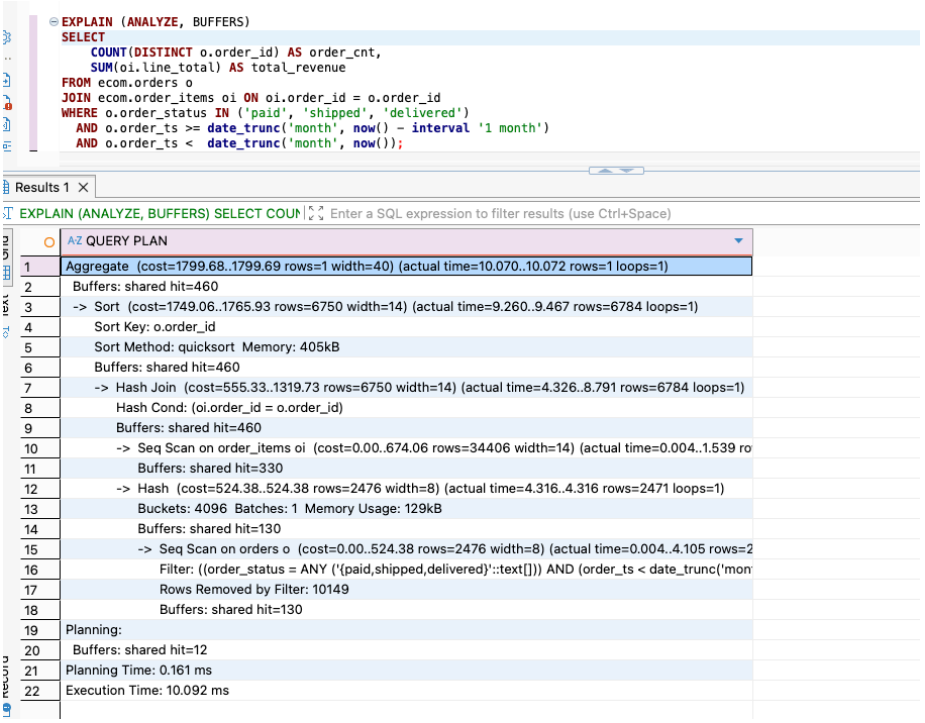
```
CREATE OR REPLACE FUNCTION ecom.safe_div(numer NUMERIC, denom NUMERIC)
RETURNS NUMERIC
LANGUAGE sql
IMMUTABLE
AS $$
    SELECT CASE
        WHEN denom IS NULL OR denom = 0 THEN NULL   -- 분모가 없거나 0이면 에러 대신 NULL 반환
        ELSE ROUND(numer / denom, 2)
    END
$$;
```



EXPLAIN ANALYZE로 병목 파악 (최소 1개)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    COUNT(DISTINCT o.order_id) AS order_cnt,
    SUM(oi.line_total) AS total_revenue
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  AND o.order_ts >= date_trunc('month', now() - interval '1 month')
  AND o.order_ts <  date_trunc('month', now());
```

결과

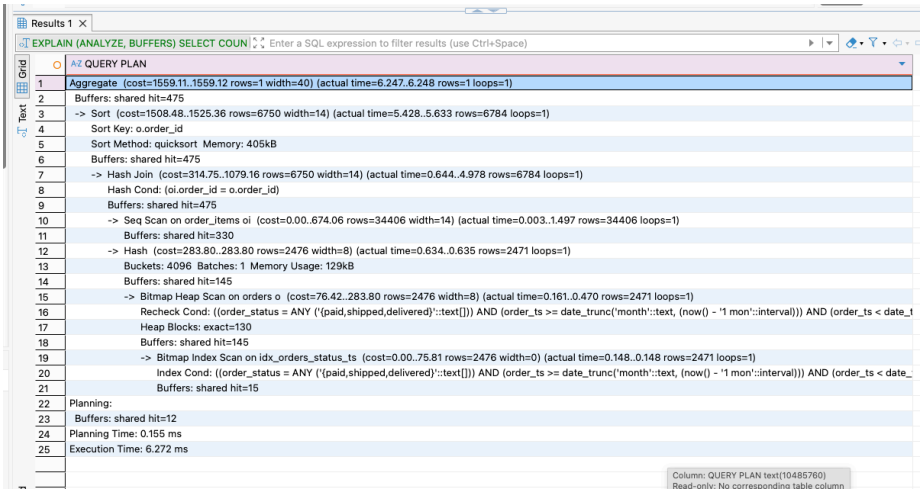


이 실행계획의 병목은 orders 테이블의 Seq Scan이라고 생각했습니다. `order_status IN (...)` 과 `order_ts` 범위 조건에 매칭되는 인덱스가 없어서 orders 전체(약 12,620행)를 풀스캔한 뒤 필터로 10,149행을 버리고 2,471행만 남기는 것을 확인했고(Rows Removed by Filter: 10149), 이 비효율적인 스캔이 이후 Hash Join과 `COUNT(DISTINCT order_id)` 에 필요한 Sort(cost 1749로 전체 노드 중 가장 높음) 비용까지 끌어올리는 구조라고 판단했습니다.

인덱스 추가/개선 (CREATE INDEX -)

위 병목의 해결책으로, 상태와 날짜를 동시에 타는 복합 인덱스를 추가했습니다.

```
CREATE INDEX IF NOT EXISTS idx_inventory_low_stock
ON ecom.inventory(product_id)
WHERE qty_on_hand <= reorder_point;
```



Join 전략 비교 : Hash Join vs Nested Loop, Bitmap Heap Scan 등

[비교 전략]

우선 세 전략을 비교하기에 앞서서, 1. 현재 기본 설정이 어떻게 되어있는지 확인했습니다.

그후, 2. 기본 실행 계획 먼저 실행해서 어떤 join 을 선택하는지 확인했습니다.

3. 그 이후에 Hash join 만 ON 으로 설정, Nested Loop 만 On 으로 설정 , Bitmap Heap Scan 만 on 으로 설정 하는 결과를 확인했습니다.

또한 4. Hash join 만 Off 했을 때 nested 와 Bitmap 둘 중에 어떤 join 으로 실행되는지, Nested Loop 만 Off 했을 때 Hash 와 Bitmap 중 어떤 join 으로 실행 되는지 ... 등을 확인해보았습니다.

1. 현재 Planner 기본 설정 확인

```
SHOW enable_hashjoin;
SHOW enable_nestloop;
SHOW enable_mergejoin;
```

모두 On 인 것을 확인했습니다.

2. 기본 실행계획

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.line_total) AS total_revenue
FROM ecom.order_items oi
JOIN ecom.orders o
    ON o.order_id = oi.order_id
JOIN ecom.products p
    ON p.product_id = oi.product_id
```

```
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY
    p.product_id,
    p.product_name;


-----

HashAggregate  (cost=1401.95..1409.45 rows=600 width=51) (actual time=15.832..15.932 rows=600
loops=1)
  Group Key: p.product_id
  Batches: 1  Memory Usage: 297kB
  Buffers: shared hit=467
  ->  Hash Join  (cost=441.57..1273.76 rows=25638 width=25) (actual time=2.327..12.060 rows=
25544 loops=1)
        Hash Cond: (oi.product_id = p.product_id)
        Buffers: shared hit=467
        ->  Hash Join  (cost=421.07..1185.48 rows=25638 width=14) (actual time=2.220..8.703 r
ows=25544 loops=1)
              Hash Cond: (oi.order_id = o.order_id)
              Buffers: shared hit=460
              ->  Seq Scan on order_items oi  (cost=0.00..674.06 rows=34406 width=22) (actual
time=0.002..1.607 rows=34406 loops=1)
                    Buffers: shared hit=330
              ->  Hash  (cost=303.52..303.52 rows=9404 width=8) (actual time=2.213..2.214 row
s=9404 loops=1)
                    Buckets: 16384  Batches: 1  Memory Usage: 496kB
                    Buffers: shared hit=130
                    ->  Seq Scan on orders o  (cost=0.00..303.52 rows=9404 width=8) (actual t
ime=0.004..1.544 rows=9404 loops=1)
                          Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                          Rows Removed by Filter: 3216
                          Buffers: shared hit=130
        ->  Hash  (cost=13.00..13.00 rows=600 width=19) (actual time=0.104..0.104 rows=600 l
oops=1)
              Buckets: 1024  Batches: 1  Memory Usage: 39kB
              Buffers: shared hit=7
              ->  Seq Scan on products p  (cost=0.00..13.00 rows=600 width=19) (actual time=
0.006..0.051 rows=600 loops=1)
                    Buffers: shared hit=7
Planning:
  Buffers: shared hit=27
Planning Time: 0.220 ms
Execution Time: 15.982 ms
```

EXPLAIN (ANALYZE, BUFFERS) S | Enter a SQL expression to filter results (use Ctrl+Space)

	QUERY PLAN
1	HashAggregate (cost=1401.95..1409.45 rows=600 width=51) (actual time=15.832..15.932 rows=600 loops=1)
2	Group Key: p.product_id
3	Batches: 1 Memory Usage: 297kB
4	Buffers: shared hit=467
5	-> Hash Join (cost=441.57..1273.76 rows=25638 width=25) (actual time=2.327..12.060 rows=25544 loops=1)
6	Hash Cond: (oi.product_id = p.product_id)
7	Buffers: shared hit=467
8	-> Hash Join (cost=421.07..1185.48 rows=25638 width=14) (actual time=2.220..8.703 rows=25544 loops=1)
9	Hash Cond: (oi.order_id = o.order_id)
10	Buffers: shared hit=460
11	-> Seq Scan on order_items oi (cost=0.00..674.06 rows=34406 width=22) (actual time=0.002..1.607 rows=
12	Buffers: shared hit=330
13	-> Hash (cost=303.52..303.52 rows=9404 width=8) (actual time=2.213..2.214 rows=9404 loops=1)
14	Buckets: 16384 Batches: 1 Memory Usage: 496kB
15	Buffers: shared hit=130
16	-> Seq Scan on orders o (cost=0.00..303.52 rows=9404 width=8) (actual time=0.004..1.544 rows=940
17	Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
18	Rows Removed by Filter: 3216
19	Buffers: shared hit=130
20	-> Hash (cost=13.00..13.00 rows=600 width=19) (actual time=0.104..0.104 rows=600 loops=1)
21	Buckets: 1024 Batches: 1 Memory Usage: 39kB
22	Buffers: shared hit=7
23	-> Seq Scan on products p (cost=0.00..13.00 rows=600 width=19) (actual time=0.006..0.051 rows=600 loo
24	Buffers: shared hit=7
25	Planning:
26	Buffers: shared hit=27
27	Planning Time: 0.220 ms
28	Execution Time: 15.982 ms

결과에 대한 간단한 분석과 생각

Buffers shared hit=467이라는 지표를 확인했을 때, 그 내역이 order_items 스캔 330, orders 스캔 130, products 스캔 7로 나뉘어 있는 것을 보았습니다.

이는 세 테이블을 각각 정확히 한 번씩만 순차적으로 읽었다는 뜻이므로, 반복적인 재방문 없이 최소한의 I/O로 조인이 끝났다고 판단했습니다.

또한 실행시간이 15.982ms로 전체 케이스 중 가장 낮게 나왔는데, orders(필터 후 9,404건)와 products(600건)라는 상대적으로 작은 테이블을 해시 테이블로 먼저 만들어두고 order_items(34,406건)를 한 번만 스캔하며 프로브하는 구조이기 때문에, 큰 테이블 쪽에서 데이터 재방문이 발생하지 않아 속도가 빠른 것으로 이해했습니다.

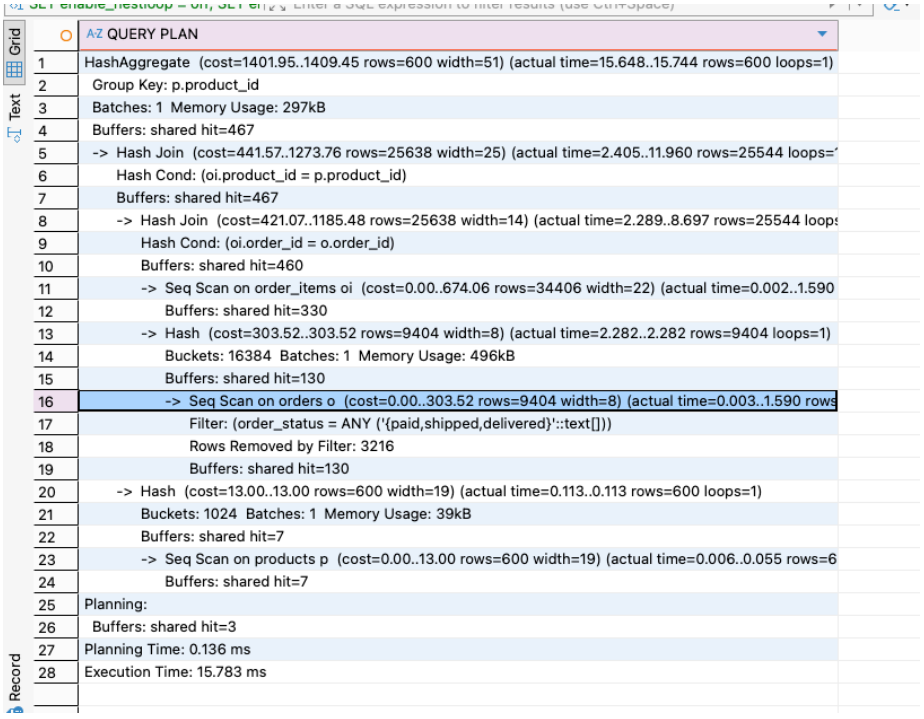
3-A. Hash Join만 강제로 사용

```
SET enable_nestloop = off;
SET enable_mergejoin = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.line_total) AS total_revenue
FROM ecom.order_items oi
JOIN ecom.orders o
    ON o.order_id = oi.order_id
JOIN ecom.products p
    ON p.product_id = oi.product_id
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY
    p.product_id,
    p.product_name;


----
HashAggregate  (cost=1401.95..1409.45 rows=600 width=51) (actual time=15.648..15.744 rows=600
loops=1)
```

```
  Group Key: p.product_id
  Batches: 1  Memory Usage: 297kB
  Buffers: shared hit=467
  ->  Hash Join  (cost=441.57..1273.76 rows=25638 width=25) (actual time=2.405..11.960 rows=
25544 loops=1)
        Hash Cond: (oi.product_id = p.product_id)
        Buffers: shared hit=467
        ->  Hash Join  (cost=421.07..1185.48 rows=25638 width=14) (actual time=2.289..8.697 r
ows=25544 loops=1)
              Hash Cond: (oi.order_id = o.order_id)
              Buffers: shared hit=460
              ->  Seq Scan on order_items oi  (cost=0.00..674.06 rows=34406 width=22) (actual
time=0.002..1.590 rows=34406 loops=1)
                    Buffers: shared hit=330
              ->  Hash  (cost=303.52..303.52 rows=9404 width=8) (actual time=2.282..2.282 row
s=9404 loops=1)
                    Buckets: 16384  Batches: 1  Memory Usage: 496kB
                    Buffers: shared hit=130
                    ->  Seq Scan on orders o  (cost=0.00..303.52 rows=9404 width=8) (actual t
ime=0.003..1.590 rows=9404 loops=1)
                          Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                          Rows Removed by Filter: 3216
                          Buffers: shared hit=130
        ->  Hash  (cost=13.00..13.00 rows=600 width=19) (actual time=0.113..0.113 rows=600 l
oops=1)
              Buckets: 1024  Batches: 1  Memory Usage: 39kB
              Buffers: shared hit=7
              ->  Seq Scan on products p  (cost=0.00..13.00 rows=600 width=19) (actual time=
0.006..0.055 rows=600 loops=1)
                    Buffers: shared hit=7
Planning:
  Buffers: shared hit=3
Planning Time: 0.136 ms
Execution Time: 15.783 ms
```



결과에 대한 간단한 분석과 생각

enable_nestloop, enable_mergejoin을 모두 off 시켰음에도 실행계획이 기본 계획과 동일하게 나온 것을 확인했습니다. 이를 통해 별도의 강제 없이도 이미 Hash Join을 최적 경로로 선택하고 있었다는 것을 알 수 있었고, 실행시간(15.783ms)과 Buffers(467)까지 기본 계획과 거의 일치하는 것을 보아 현재 데이터 분포에서는 Hash Join이 우위를 갖는다는 점을 재확인했습니다.

3-B. Nested Loop만 강제로 사용

```
SET enable_hashjoin = off;
SET enable_mergejoin = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.line_total) AS total_revenue
FROM ecom.order_items oi
JOIN ecom.orders o
    ON o.order_id = oi.order_id
JOIN ecom.products p
    ON p.product_id = oi.product_id
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY
    p.product_id,
    p.product_name;

------

GroupAggregate  (cost=20000000000.86..20000007698.17 rows=600 width=51) (actual time=0.107..34.307 rows=600 loops=1)
  Group Key: p.product_id
  Buffers: shared hit=49374
  ->  Nested Loop  (cost=20000000000.86..20000007562.48 rows=25638 width=25) (actual time=0.029..31.751 rows=25544 loops=1)
        Buffers: shared hit=49374
        ->  Nested Loop  (cost=10000000000.57..10000002633.68 rows=34406 width=33) (actual time=0.014..9.894 rows=34406 loops=1)
              Buffers: shared hit=11514
              ->  Index Scan using products_pkey on products p  (cost=0.28..35.27 rows=600 width=19) (actual time=0.007..0.092 rows=600 loops=1)
                    Buffers: shared hit=10
              ->  Index Scan using idx_order_items_product on order_items oi  (cost=0.29..3.76 rows=57 width=22) (actual time=0.001..0.011 rows=57 loops=600)
                    Index Cond: (product_id = p.product_id)
                    Buffers: shared hit=11504
        ->  Memoize  (cost=0.30..0.34 rows=1 width=8) (actual time=0.001..0.001 rows=1 loops=34406)
              Cache Key: oi.order_id
              Cache Mode: logical
              Hits: 21786  Misses: 12620  Evictions: 0  Overflows: 0  Memory Usage: 1255kB
              Buffers: shared hit=37860
              ->  Index Scan using orders_pkey on orders o  (cost=0.29..0.33 rows=1 width=8) (actual time=0.001..0.001 rows=1 loops=12620)
                    Index Cond: (order_id = oi.order_id)
                    Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
```

```
                    Rows Removed by Filter: 0
                    Buffers: shared hit=37860
Planning:
  Buffers: shared hit=3
Planning Time: 0.143 ms
Execution Time: 34.341 ms
```

결과에 대한 간단한 분석과 생각

가장 먼저 눈에 띈 지표는 cost 값이 200000000000이라는 비정상적으로 큰 숫자로 나온 부분이었습니다.

이를 보고 사실상 "정상적으로는 선택하지 말라"는 페널티를 매겨두었다는 것을 확인했고, 강제로 켰을 때만 등장한다는 사실을 통해 이 경로가 원래는 비효율적이라 자동으로 배제되는 경로라는 것을 이해했습니다.

또한 Buffers shared hit이 49,374로 다른 모든 케이스보다 압도적으로 높게 나왔는데, Index Scan on orders_pkey가 loops=12,620회 반복되면서 buffers 37,860을 차지하고 있는 것을 확인했습니다.

이를 통해 order_items의 각 행마다 orders 테이블을 개별적으로 재탐색하는 구조 때문에 같은 성격의 데이터를 반복해서 읽어들이는 비효율이 발생했다고 판단했습니다.

Memoize의 Hits 21,786 / Misses 12,620이라는 지표도 함께 보았는데, distinct order_id 종류가 많아 캐시 재사용률이 충분히 확보되지 않았고, 그 결과 실행시간도 34.341ms로 전체 케이스 중 가장 느리게 나온 것으로 이해했습니다.

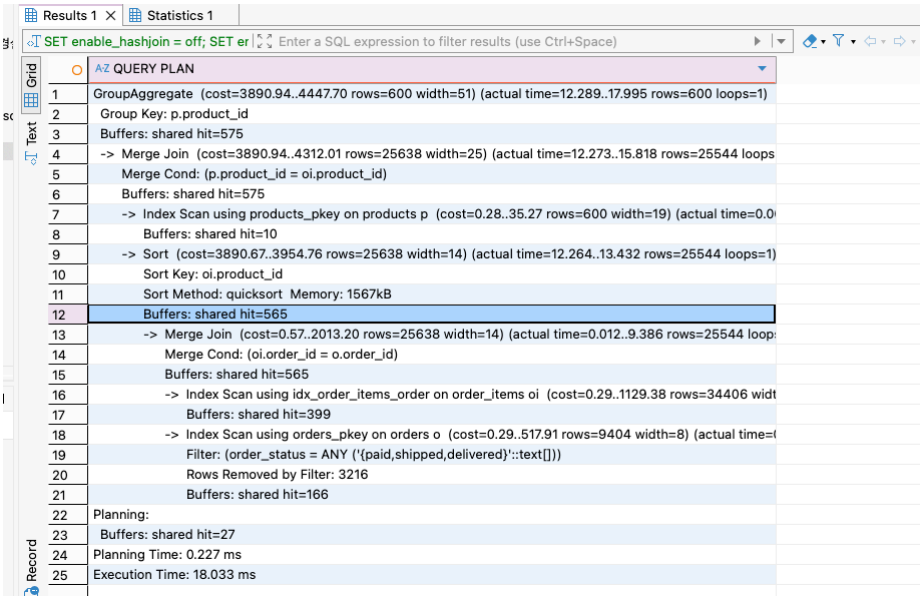
3-C. Merge Join만 강제로 사용

```
SET enable_hashjoin = off;
SET enable_nestloop = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.line_total) AS total_revenue
FROM ecom.order_items oi
JOIN ecom.orders o
    ON o.order_id = oi.order_id
JOIN ecom.products p
    ON p.product_id = oi.product_id
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY
    p.product_id,
    p.product_name;


---
GroupAggregate  (cost=3890.94..4447.70 rows=600 width=51) (actual time=12.289..17.995 rows=600 loops=1)
  Group Key: p.product_id
  Buffers: shared hit=575
  ->  Merge Join  (cost=3890.94..4312.01 rows=25638 width=25) (actual time=12.273..15.818 rows=25544 loops=1)
        Merge Cond: (p.product_id = oi.product_id)
        Buffers: shared hit=575
        ->  Index Scan using products_pkey on products p  (cost=0.28..35.27 rows=600 width=19) (actual time=0.005..0.069 rows=600 loops=1)
              Buffers: shared hit=10
        ->  Sort  (cost=3890.67..3954.76 rows=25638 width=14) (actual time=12.264..13.432 row
```

```
s=25544 loops=1)
              Sort Key: oi.product_id
              Sort Method: quicksort  Memory: 1567kB
              Buffers: shared hit=565
              ->  Merge Join  (cost=0.57..2013.20 rows=25638 width=14) (actual time=0.012..9.
386 rows=25544 loops=1)
                    Merge Cond: (oi.order_id = o.order_id)
                    Buffers: shared hit=565
                    ->  Index Scan using idx_order_items_order on order_items oi  (cost=0.2
9..1129.38 rows=34406 width=22) (actual time=0.005..2.942 rows=34406 loops=1)
                          Buffers: shared hit=399
                    ->  Index Scan using orders_pkey on orders o  (cost=0.29..517.91 rows=940
4 width=8) (actual time=0.006..2.082 rows=9404 loops=1)
                          Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                          Rows Removed by Filter: 3216
                          Buffers: shared hit=166
Planning:
  Buffers: shared hit=27
Planning Time: 0.227 ms
Execution Time: 18.033 ms
```



결과에 대한 간단한 분석과 생각

Sort 노드가 별도로 추가된 것을 확인했습니다.

Sort Method: quicksort, Memory: 1567kB라는 지표를 보아, order_items가 product_id 기준으로 정렬되어 있지 않았기 때문에 조인 전에 별도의 정렬 작업이 필요했다는 것을 이해했습니다.

반면 products_pkey, idx_order_items_order 인덱스를 사용한 부분은 Index Scan만으로 이미 정렬된 순서를 얻을 수 있어 추가 정렬이 없었던 점도 확인했습니다.

실행시간은 18.033ms로 Hash Join보다는 근소하게 느리고 Nested Loop보다는 훨씬 빨랐는데, 이 결과를 통해 조인 키에 인덱스가 잘 갖춰져 있으면 Merge Join도 정렬 비용을 감안하더라도 충분히 경쟁력 있는 대안이 될 수 있다고 판단했습니다.

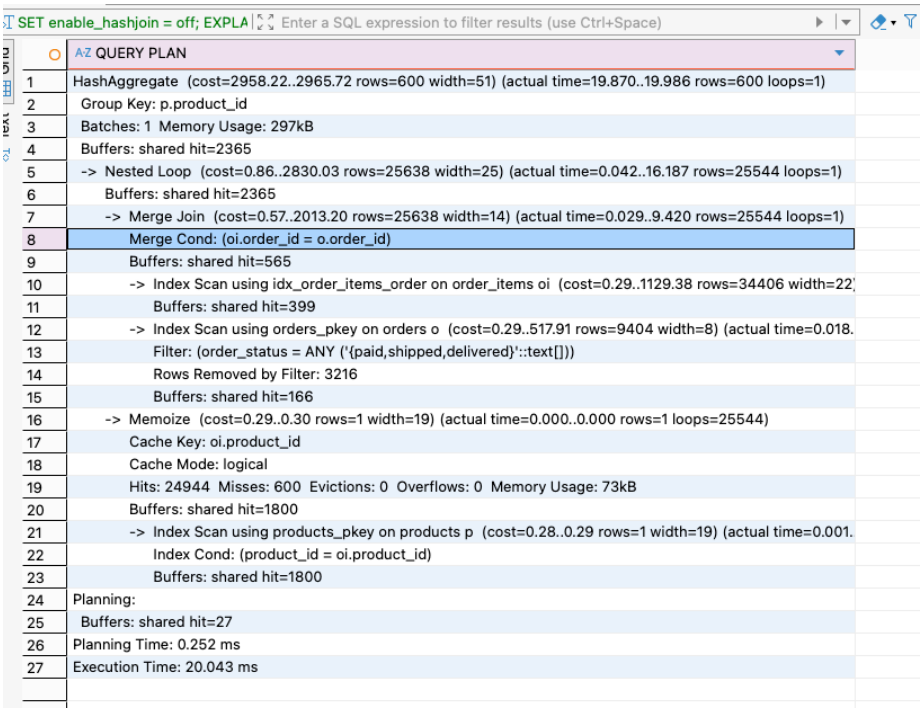
4-A. Hash Join만 사용 금지 → Nested Loop 또는 Merge Join 선택

```
SET enable_hashjoin = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.line_total) AS total_revenue
FROM ecom.order_items oi
JOIN ecom.orders o
    ON o.order_id = oi.order_id
JOIN ecom.products p
    ON p.product_id = oi.product_id
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY
    p.product_id,
    p.product_name;

---

HashAggregate  (cost=2958.22..2965.72 rows=600 width=51) (actual time=19.870..19.986 rows=600
loops=1)
  Group Key: p.product_id
  Batches: 1  Memory Usage: 297kB
  Buffers: shared hit=2365
  ->  Nested Loop  (cost=0.86..2830.03 rows=25638 width=25) (actual time=0.042..16.187 rows=
25544 loops=1)
        Buffers: shared hit=2365
        ->  Merge Join  (cost=0.57..2013.20 rows=25638 width=14) (actual time=0.029..9.420 ro
ws=25544 loops=1)
              Merge Cond: (oi.order_id = o.order_id)
              Buffers: shared hit=565
              ->  Index Scan using idx_order_items_order on order_items oi  (cost=0.29..1129.
38 rows=34406 width=22) (actual time=0.009..2.974 rows=34406 loops=1)
                    Buffers: shared hit=399
              ->  Index Scan using orders_pkey on orders o  (cost=0.29..517.91 rows=9404 widt
h=8) (actual time=0.018..2.138 rows=9404 loops=1)
                    Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                    Rows Removed by Filter: 3216
                    Buffers: shared hit=166
        ->  Memoize  (cost=0.29..0.30 rows=1 width=19) (actual time=0.000..0.000 rows=1 loops
=25544)
              Cache Key: oi.product_id
              Cache Mode: logical
              Hits: 24944  Misses: 600  Evictions: 0  Overflows: 0  Memory Usage: 73kB
              Buffers: shared hit=1800
              ->  Index Scan using products_pkey on products p  (cost=0.28..0.29 rows=1 width
=19) (actual time=0.001..0.001 rows=1 loops=600)
                    Index Cond: (product_id = oi.product_id)
                    Buffers: shared hit=1800
Planning:
  Buffers: shared hit=27
Planning Time: 0.252 ms
Execution Time: 20.043 ms
```



SET enable_hashjoin = off; EXPLA | Enter a SQL expression to filter results (use Ctrl+Space)

	QUERY PLAN
1	HashAggregate (cost=2958.22..2965.72 rows=600 width=51) (actual time=19.870..19.986 rows=600 loops=1)
2	Group Key: p.product_id
3	Batches: 1 Memory Usage: 297kB
4	Buffers: shared hit=2365
5	-> Nested Loop (cost=0.86..2830.03 rows=25638 width=25) (actual time=0.042..16.187 rows=25544 loops=1)
6	Buffers: shared hit=2365
7	-> Merge Join (cost=0.57..2013.20 rows=25638 width=14) (actual time=0.029..9.420 rows=25544 loops=1)
8	Merge Cond: (oi.order_id = o.order_id)
9	Buffers: shared hit=565
10	-> Index Scan using idx_order_items_order on order_items oi (cost=0.29..1129.38 rows=34406 width=22
11	Buffers: shared hit=399
12	-> Index Scan using orders_pkey on orders o (cost=0.29..517.91 rows=9404 width=8) (actual time=0.018.
13	Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
14	Rows Removed by Filter: 3216
15	Buffers: shared hit=166
16	-> Memoize (cost=0.29..0.30 rows=1 width=19) (actual time=0.000..0.000 rows=1 loops=25544)
17	Cache Key: oi.product_id
18	Cache Mode: logical
19	Hits: 24944 Misses: 600 Evictions: 0 Overflows: 0 Memory Usage: 73kB
20	Buffers: shared hit=1800
21	-> Index Scan using products_pkey on products p (cost=0.28..0.29 rows=1 width=19) (actual time=0.001.
22	Index Cond: (product_id = oi.product_id)
23	Buffers: shared hit=1800
24	Planning:
25	Buffers: shared hit=27
26	Planning Time: 0.252 ms
27	Execution Time: 20.043 ms

결과에 대한 간단한 분석과 생각

order_items와 orders는 여전히 Merge Join으로, products는 Nested Loop + Memoize로 처리되는 혼합 플랜이 나온 것을 확인했습니다.

특히 Memoize의 Hits: 24944, Misses: 600이라는 지표에서 Misses 값이 products 테이블의 전체 행 수(600)와 정확히 일치하는 것을 보고, distinct key 개수만큼만 캐시 미스가 발생하고 나머지는 모두 캐시로 처리되었다는 것을 알았습니다.

이를 통해 반복되는 조인 키의 종류가 적을 때는 Nested Loop라도 Memoize가 사실상 해시 테이블과 비슷한 역할을 해줄 수 있다고 이해했습니다. 다만 Buffers shared hit이 2,365로 Hash Join 기본 플랜(467) 대비 5배 이상 높게 나온 것을 보아, Nested Loop는 25,544번 반복(loops)마다 개별 탐색을 수행하는 구조라서 캐시가 있어도 순차 스캔 대비 오버헤드가 남는다는 것을 확인했습니다.

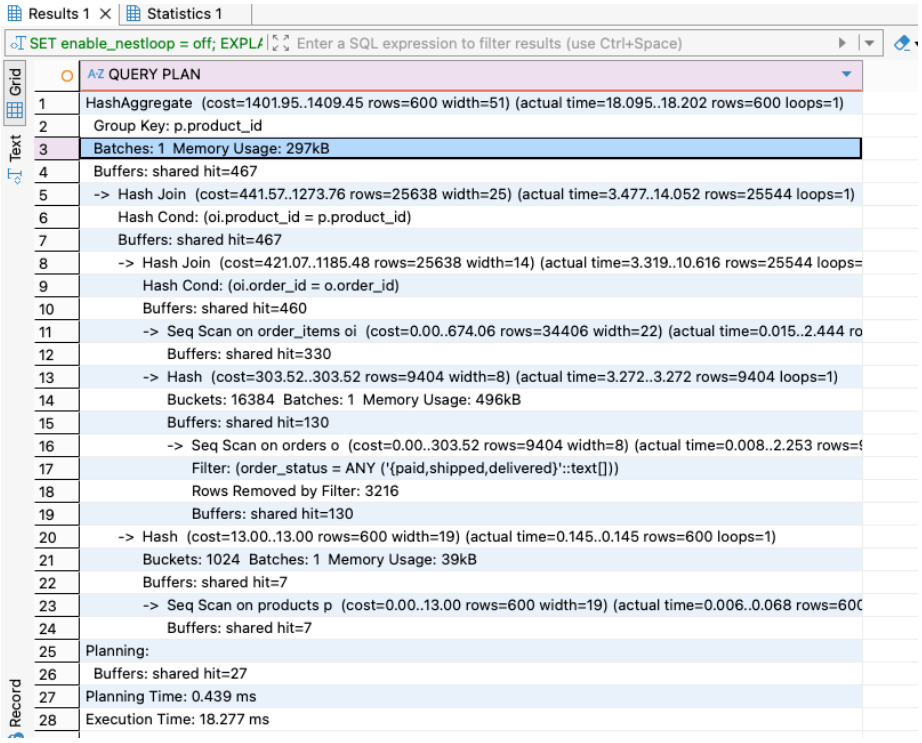
4-B. Nested Loop만 사용 금지 → Hash Join 또는 Merge Join 선택

```
SET enable_nestloop = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.line_total) AS total_revenue
FROM ecom.order_items oi
JOIN ecom.orders o
    ON o.order_id = oi.order_id
JOIN ecom.products p
    ON p.product_id = oi.product_id
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY
    p.product_id,
    p.product_name;

    ----
```

```
HashAggregate  (cost=1401.95..1409.45 rows=600 width=51) (actual time=18.095..18.202 rows=600
loops=1)
  Group Key: p.product_id
  Batches: 1  Memory Usage: 297kB
  Buffers: shared hit=467
  ->  Hash Join  (cost=441.57..1273.76 rows=25638 width=25) (actual time=3.477..14.052 rows=
25544 loops=1)
        Hash Cond: (oi.product_id = p.product_id)
        Buffers: shared hit=467
        ->  Hash Join  (cost=421.07..1185.48 rows=25638 width=14) (actual time=3.319..10.616
rows=25544 loops=1)
              Hash Cond: (oi.order_id = o.order_id)
              Buffers: shared hit=460
              ->  Seq Scan on order_items oi  (cost=0.00..674.06 rows=34406 width=22) (actual
time=0.015..2.444 rows=34406 loops=1)
                    Buffers: shared hit=330
              ->  Hash  (cost=303.52..303.52 rows=9404 width=8) (actual time=3.272..3.272 row
s=9404 loops=1)
                    Buckets: 16384  Batches: 1  Memory Usage: 496kB
                    Buffers: shared hit=130
                    ->  Seq Scan on orders o  (cost=0.00..303.52 rows=9404 width=8) (actual t
ime=0.008..2.253 rows=9404 loops=1)
                          Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                          Rows Removed by Filter: 3216
                          Buffers: shared hit=130
        ->  Hash  (cost=13.00..13.00 rows=600 width=19) (actual time=0.145..0.145 rows=600 l
oops=1)
              Buckets: 1024  Batches: 1  Memory Usage: 39kB
              Buffers: shared hit=7
              ->  Seq Scan on products p  (cost=0.00..13.00 rows=600 width=19) (actual time=
0.006..0.068 rows=600 loops=1)
                    Buffers: shared hit=7
Planning:
  Buffers: shared hit=27
Planning Time: 0.439 ms
Execution Time: 18.277 ms
```



결과에 대한 간단한 분석과 생각

실행계획이 기본 플랜과 동일하게 Hash Join으로 나온 것을 확인했습니다.

이를 통해 기본 플랜에는 애초에 Nested Loop가 포함되어 있지 않았기 때문에 이 옵션을 꺼도 영향이 없었다는 것을 이해했습니다. 실행시간이 18.277ms로 기본 플랜(15.982ms)보다 약간 높게 나왔지만 Buffers는 467로 동일했는데, 플랜 구조 차이가 아니라 측정 시점의 오차 범위 내 변동으로 판단했습니다.

4-C. Merge Join만 사용 금지

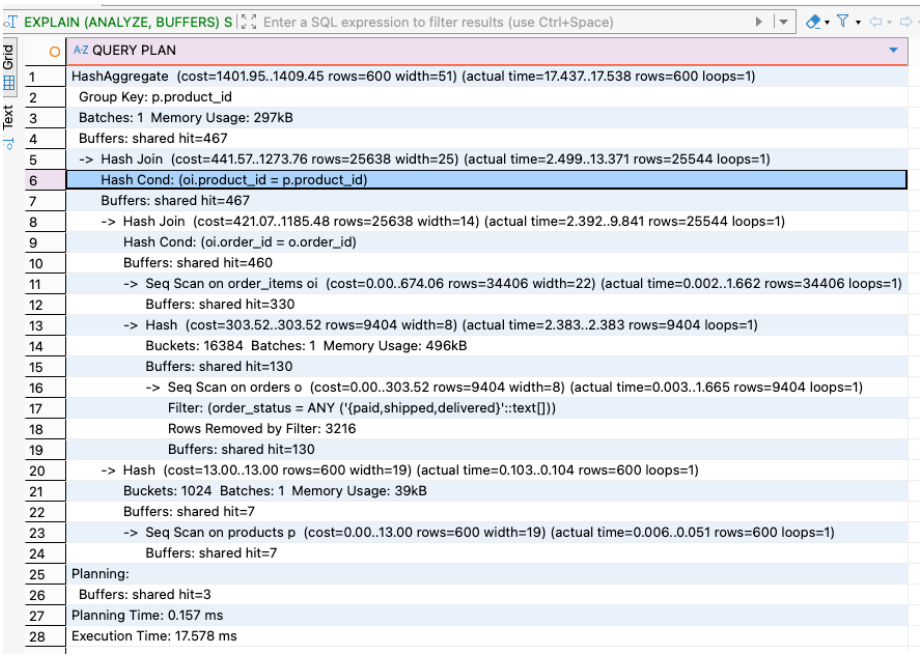
```
SET enable_mergejoin = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.line_total) AS total_revenue
FROM ecom.order_items oi
JOIN ecom.orders o
    ON o.order_id = oi.order_id
JOIN ecom.products p
    ON p.product_id = oi.product_id
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY
    p.product_id,
    p.product_name;

------

HashAggregate  (cost=1401.95..1409.45 rows=600 width=51) (actual time=17.437..17.538 rows=600
loops=1)
  Group Key: p.product_id
```

```
  Batches: 1  Memory Usage: 297kB
  Buffers: shared hit=467
  ->  Hash Join  (cost=441.57..1273.76 rows=25638 width=25) (actual time=2.499..13.371 rows=
25544 loops=1)
        Hash Cond: (oi.product_id = p.product_id)
        Buffers: shared hit=467
        ->  Hash Join  (cost=421.07..1185.48 rows=25638 width=14) (actual time=2.392..9.841 r
ows=25544 loops=1)
              Hash Cond: (oi.order_id = o.order_id)
              Buffers: shared hit=460
              ->  Seq Scan on order_items oi  (cost=0.00..674.06 rows=34406 width=22) (actual
time=0.002..1.662 rows=34406 loops=1)
                    Buffers: shared hit=330
              ->  Hash  (cost=303.52..303.52 rows=9404 width=8) (actual time=2.383..2.383 row
s=9404 loops=1)
                    Buckets: 16384  Batches: 1  Memory Usage: 496kB
                    Buffers: shared hit=130
                    ->  Seq Scan on orders o  (cost=0.00..303.52 rows=9404 width=8) (actual t
ime=0.003..1.665 rows=9404 loops=1)
                          Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                          Rows Removed by Filter: 3216
                          Buffers: shared hit=130
        ->  Hash  (cost=13.00..13.00 rows=600 width=19) (actual time=0.103..0.104 rows=600 l
oops=1)
              Buckets: 1024  Batches: 1  Memory Usage: 39kB
              Buffers: shared hit=7
              ->  Seq Scan on products p  (cost=0.00..13.00 rows=600 width=19) (actual time=
0.006..0.051 rows=600 loops=1)
                    Buffers: shared hit=7
Planning:
  Buffers: shared hit=3
Planning Time: 0.157 ms
Execution Time: 17.578 ms
```



이 경우 역시 기본 플랜과 동일한 Hash Join 구조가 그대로 나온 것을 확인했습니다.

이를 통해 기본 플랜에 Merge Join도 포함되어 있지 않았다는 것을 다시 확인할 수 있었고, 결국 이 쿼리는 Nested Loop·Merge Join 옵션의 활성화 여부와 무관하게 Hash Join이 가장 비용이 낮은 경로로 선택된다는 결론을 내렸습니다.

5. 실행 결과 요약

케이스	실제 플랜	실행시간	Buffers(hit)
기본 (전부 ON)	Hash Join × 2	15.98ms	467
Hash Join 강제	Hash Join × 2 (동일)	15.78ms	467
Nested Loop 강제	Nested Loop + Memoize	34.34ms	**49,374**
Merge Join 강제	Merge Join × 2 (+Sort)	18.03ms	575
hashjoin=off	Merge Join + Nested Loop(Memoize)	20.04ms	2,365
nestloop=off	Hash Join × 2 (기본과 동일)	18.28ms	467
mergejoin=off	Hash Join × 2 (기본과 동일)	17.58ms	467

Materialized View 활용 : mv_daily_gmv로 리포트 질의 가속, 갱신 전략 검토

요구조건

- 매일매일 총 판매금액을 조회할 수 있게 단, 매번 JOIN 해서 SUM 하면 속도가 느리므로 Materialized View 사용.
- mv_daily_gmv라는 Materialized View를 만들어서 리포트 쿼리를 빠르게 실행하고, 데이터가 얼마나 자주 바뀌는지에 맞춰 갱신 주기를 설계 (오후 3시 기준으로 설정)

Materialized View 정의

```
CREATE MATERIALIZED VIEW ecom.mv_daily_gmv AS
SELECT
    o.order_ts::date            AS sale_date,
    COUNT(DISTINCT o.order_id)    AS order_count,
    SUM(oi.line_total)            AS total_gmv
FROM ecom.order_items oi
JOIN ecom.orders o
    ON o.order_id = oi.order_id
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY o.order_ts::date
WITH DATA;

-- REFRESH CONCURRENTLY를 쓰려면 유니크 인덱스가 반드시 필요
CREATE UNIQUE INDEX idx_mv_daily_gmv_sale_date
    ON ecom.mv_daily_gmv (sale_date);
```

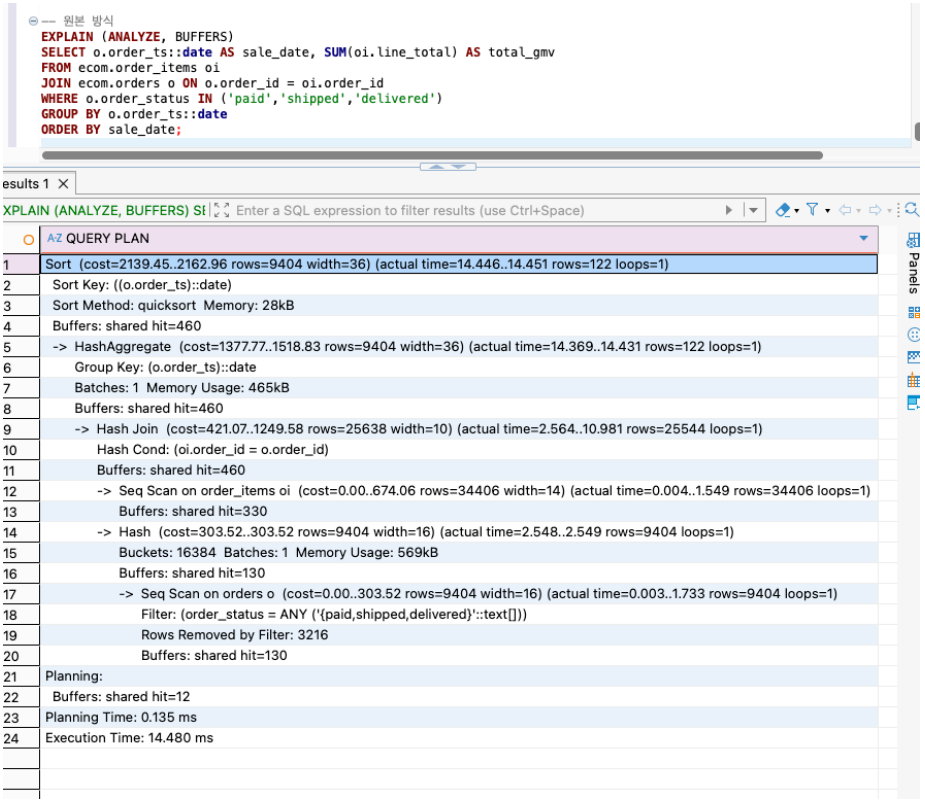
CONCURRENTLY 옵션 없이 REFRESH를 실행하면 갱신 도중 MV 전체에 락이 걸려 그 사이 리포트 조회가 막힌다는 것을 확인했습니다. 이를 막기 위해서는 유니크 인덱스가 있어야만 CONCURRENTLY 갱신이 가능하다는 PostgreSQL의 제약 조건을 확인했고, sale_date를 유니크 키로 잡는 것이 자연스럽다고 판단하여 위와 같이 인덱스를 함께 설계했습니다.

리포트 조회 쿼리

```
-- 원본 방식 (매번 JOIN + SUM, 느림)
EXPLAIN (ANALYZE, BUFFERS)
SELECT o.order_ts::date AS sale_date, SUM(oi.line_total) AS total_gmv
```

```sql
FROM ecom.order_items oi
JOIN ecom.orders o ON o.order_id = oi.order_id
WHERE o.order_status IN ('paid','shipped','delivered')
GROUP BY o.order_ts::date
ORDER BY sale_date;

-- MV 활용 방식 (사전 집계된 결과만 조회, 빠름)
EXPLAIN (ANALYZE, BUFFERS)
SELECT sale_date, order_count, total_gmv
FROM ecom.mv_daily_gmv
ORDER BY sale_date;
```

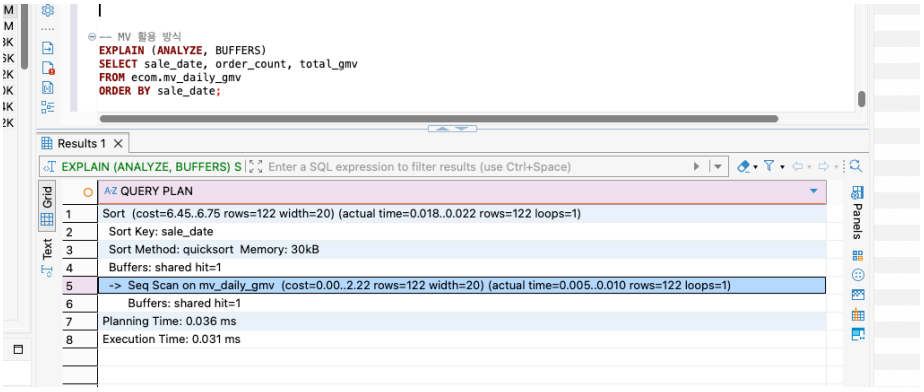


→ 원본 방식

Buffers shared hit=460, 실행시간 14.480ms가 나온 것을 확인했습니다. order_items 스캔에 330, orders 스캔에 130이 들어간 것으로 보아 매 조회마다 34,406건짜리 order_items 전체를 훑고 orders와 해시 조인을 한 뒤 다시 정렬까지 해야 하는 구조라고 생각했습니다.

HashAggregate의 추정 rows=9404 대비 실제 rows=122라는 차이도 눈에 띄었는데, `order_ts::date`처럼 표현식으로 그룹핑되는 컬럼은 플래너가 distinct 값 개수를 알 수 없어 필터링된 orders 행 수를 그대로 추정치로 쓴 것으로 이해했습니다.

MV 활용



→ Buffers shared hit=1, 실행시간 0.042ms가 나온 것을 확인했습니다.

원본 방식(Buffers=460, 14.480ms)과 비교했을 때 Buffers는 460배, 실행시간은 약 345배 차이가 났는데, 이는 MV가 이미 122행짜리 결과 테이블로 접혀 있어서 Seq Scan on mv_daily_gmv 하나만으로 조회가 끝나기 때문이라고 생각했습니다.
원본 방식에서는 order_items(330 buffers)와 orders(130 buffers)를 매번 스캔하고 Hash Join까지 거쳐야 했지만, MV 쪽은 그 조인 자체가 아예 발생하지 않는다는 것을 실행계획에 Hash Join, Seq Scan on order_items/orders 같은 노드가 전혀 없다는 점으로 확인할 수 있었습니다. rows=122라는 실제 결과 건수도 원본 방식의 실제 rows=122와 정확히 일치해서, MV가 원본과 동일한 데이터를 담고 있다는 것도 함께 확인했습니다.

방식	Buffers	Execution Time
원본 (JOIN+SUM)	460	14.480ms
MV 활용	1	0.042ms

갱신 전략: 오후 3시 기준

GMV는 "하루 단위 누적 총액"이라는 특성상 분 단위로 실시간 반영될 필요는 없다고 생각했습니다. 다만 리포트 소비 시점(예: 오후 업무 보고)에 맞춰 오후 3시에는 최신 데이터가 반영되어 있어야 한다는 요건을 확인했고, 이에 따라 매일 오후 3시에 1회 갱신하는 배치 전략을 설계했습니다.

```
REFRESH MATERIALIZED VIEW CONCURRENTLY ecom.mv_daily_gmv;
```

```
-- pg_cron 확장을 사용하는 경우
SELECT cron.schedule(
    'refresh_mv_daily_gmv',
    '0 15 * * *',    -- 매일 오후 3시
    $$REFRESH MATERIALIZED VIEW CONCURRENTLY ecom.mv_daily_gmv$$
);
```

다만 실제로 `cron` 스키마 존재 여부를 확인해본 결과, 현재 세팅된 환경에는 pg_cron 확장이 설치되어 있지 않아 위 방식은 바로 사용할 수 없다는 것을 확인했습니다.

pg_cron은 서버 설 (`shared_preload_libraries`) 변경과 재시작이 필요한 확장이라 별도 관리자 권한 없이는 즉시 켤 수 없는 구조였습니다.

이런 제약이 있는 환경에서는 OS 레벨 cron이나 Airflow 같은 외부 스케줄러에서 동일한 REFRESH 문을 오후 3시에 실행하도록 등록하는 방식으로도 동일한 효과를 낼 수 있다는 것을 확인했고, 이번 설계에서는 이 방식을 대안으로 채택했습니다.

(Option) 엔진별 특징 정리: Postgres / MySQL / Oracle / SQL Server 옵티마이저 차이

PostgreSQL

비용 기반 옵티마이저(Cost-Based Optimizer)를 사용하며, 통계는 ANALYZE로 수집한 MCV(Most Common Values), 히스토그램, 확장 통계(다중 컬럼 간 상관관계, 함수 종속성)를 기반으로 카디널리티를 추정한다. Hash Join, Nested Loop, Merge Join을 모두 지원하고 PostgreSQL 14부터 Memoize 노드가 추가되어 Nested Loop의 반복 탐색 비용을 캐싱으로 줄일 수 있다. 다만 카디널리티 추정치가 실제 값과 수 배~수십 배 차이 나는 경우가 있다고 보고되며, 실행 중 계획을 바꾸는 적응형 재최적화 기능은 없다(플랜은 실행 전 확정). 대신 pg_hint_plan 같은 확장이나 enable_* 파라미터로 수동 개입하는 방식이 일반적이다.

MySQL (8.0 기준)

전통적으로 Nested Loop Join(Block Nested Loop 포함)이 기본이었으나, 8.0.18부터 인덱스가 없거나 비효율적인 equi-join에 한해 Hash Join이 도입되었다. Merge Join은 지원하지 않는다. 8.0에서 컬럼 히스토그램이 추가되어 인덱스가 없는 컬럼에 대해서도 값 분포 기반 카디널리티 추정이 가능해졌지만, 인덱스 통계(cardinality)에 대한 의존도가 상대적으로 높은 편이다. 옵티마이저 힌트(/*+ HASH_JOIN(...) */, JOIN_ORDER 등)로 플랜을 직접 제어할 수 있다.

Oracle

CBO(Cost-Based Optimizer)가 디스크 I/O, CPU, 메모리 사용량을 비용 단위로 환산해 최저 비용 플랜을 선택한다. 4개 엔진 중 런타임 적응 기능이 가장 발달해 있는데, 12c부터 도입된 Adaptive Query Optimization이 대표적이다. 여기에는 실행 중 실제 로우 수를 관찰해 조인 방식을 바꾸는 Adaptive Plans, 통계가 부족할 때 즉석에서 샘플링하는 Dynamic Statistics(구 Dynamic Sampling), 실행 결과를 다음 실행에 반영하는 SQL Plan Baseline·Cardinality Feedback이 포함된다. 즉 "계획을 세우고 그대로 실행"하는 다른 엔진과 달리, 실행 중 관찰치를 바탕으로 계획 자체를 교정하는 것이 특징이다.

SQL Server

카디널리티 추정기(Cardinality Estimator, CE)가 Legacy CE(SQL Server 7.0~2012, 호환성 수준 110 이하)와 New CE(2014~, 호환성 수준 120 이상)로 나뉜다. Legacy CE는 술어 간 독립성·균등 분포·Simple Containment(조인 키가 양쪽에 항상 존재한다고 가정)를 전제하는 반면, New CE는 술어 간 상관관계를 더 반영하고 Base Containment 방식을 쓴다. 두 모델은 가정이 달라 같은 쿼리에도 다른 카디널리티 추정치와 플랜을 낼 수 있어, 호환성 수준 변경 시 성능 회귀가 발생하기도 한다. 2022(호환성 수준 160)부터는 Intelligent Query Processing의 일부로 Cardinality Estimation Feedback이 추가되어, 반복 실행되는 쿼리의 추정 오차를 관찰해 Query Store에 보정값을 저장하고 이후 실행에 반영한다.